

又过一学期，编译也结束了，计算机本科三件套也全部学完了。编译器是我目前所有的项目中最耗费心力的一个项目。

我对自己写编译器的要求只有一点：减少样板代码（reduce boilerplate）。英文是我期末的时候才了解到的词语，而中文是我前一个暑假在华为实习的时候上司对我们的要求。在华为实习做了仓颉相关的工程，确实让我感受到代码优美整洁之美，这点极大影响了我这学期写编译器的目标。总之，编译器项目还是让我感受到了代码之美。

为了追求代码质量，从语法分析开始的作业都交得时间非常极限，一周基本上要重构两次整个项目，调整目录结构、调整实现方式等等……我认为比较大的麻烦就是一开始从零实现的时候，我不知道当前这个阶段的代码要做到什么地步。我会去看往届学长的课设代码或者参赛作品，大概了解他们每个阶段的完成度，但总有地方让我觉得不满意。去看 llvm 的 clang 实现又让我觉得有点过于困难了。所以从零实现总是一步一步摸索过来的，所以我前期总是在对我的代码进行大幅重构。

在具体实现上，尽管在 Java 上实现的声明式语法分析器都是丑陋的嵌套的括号，我还是为自己想到这样简单明了的设计感到高兴。虽然 CST 到 AST 的转换我改了一遍又一遍，那种每次推倒重来的感觉让我痛苦不堪，但目前这部分的实现还是足以让我感到满意。

当然，这一目标到最后还是没有完全实现。关于减少样板代码的要求，我至今没有明白的一点是前中后端数据结构的复用性。如果分开写那么代码量就膨胀了，合起来写代码耦合度就高了。比如符号表的概念，前端和后端实际上都有，但其实是两个不同的数据载体。以及关于错误处理部分，我还在思考如何将文档中定义的错误和未定义的错误统一到一起，能否使用统一的异常处理的数据结构和方式来对待他们。

到最后竞速排序的时候还是没有守住代码质量这一大关。竞速排序最后的成绩是让我很满意的，然而代码却让我不能满意。到了后期实在是来不及了，使用 AI 辅助编程的感觉特别爽快，但却突然有种成瘾的警觉。作为对比，在写竞速排序所有的优化之前，我的代码是刚刚超过 4000 行；写完竞速排序的优化之后，代码超过了 10000 行。

让我觉得遗憾的就是没有时间把握好竞速排序之后，代码优化相关的代码质量。虽然结构上我的代码很清楚，但其实中端的各个优化的耦合度非常高，往往注释掉一个 Pass 就会造成代码编译错误。做竞速排序的时候总是容易被排行榜所左右，让我忘记我真正想要追求的代码质量的目标。

测试环节也是我觉得做的不够好的地方，虽然我有完整的测试框架，但是并行执行存在 bug 让我难以释怀。对测试用例质量的把控我也没有做好，都是收集其他人造的样例自己无脑使用。

我本来就不大喜欢 C++。国庆节一开始写了一版 C++ 的编译器实现，然后果断转成了 Java。然而一学期的 Java 实现让我也不怎么喜欢 Java 了。寒假里打算使用仓颉语言来实现一遍 SysY 编译器，既然编译的课程组老师们在教仓颉，也许有机会未来编译课程支持使用仓颉编写呢。